

Contents

Commands (C)

intro	Introduces XENIX commands.
accept, reject	Allows/prevents print requests.
acctcom	Searches for and prints process accounting files.
accton	Turns on accounting.
asktime	Sets/displays the correct time of day.
assign, deassign	Assigns and deassigns devices.
at, batch	Executes commands at a later time.
awk	Searches for and processes a pattern in a file.
backup	Performs incremental file system backup.
banner	Prints large letters.
basename	Removes directory names from pathnames.
bc	Invokes a calculator.
bdiff	Compares files too large for <i>diff</i> .
bfs	Scans big files.
cal	Prints a calendar.
calendar	Invokes a reminder service.
cat	Concatenates and displays files.
cd	Changes working directory.
chgrp	Changes group ID.
chmod	Changes the access permissions of a file or directory.
chown	Changes owner ID.
chroot	Changes root directory for command.
chsh	Changes default shell.
chswap	Changes the swap space size of the XENIX kernel.
clear, cls	Clears the terminal screen.
clri	Clears inode.
cmp	Compares two files.
comm	Selects or rejects lines common to two sorted files.
copy	Copies groups of files.
cp	Copies files.
cpio	Copies file archives in and out.
cron	Executes commands at specified times.
crontab	Manages user crontab files.
csplit	Splits files according to context.
cu	Calls another XENIX system.
date	Prints and sets the date.
dc	Invokes an arbitrary precision calculator.

Commands (C)

dd	Converts and copies a file.
devnm	Identifies device name.
df	Reports the number of free disk blocks.
diff	Compares two text files.
diff3	Compares three files.
dircmp	Compares directories.
dirname	Delivers directory part of pathname.
disable	Turns off terminals.
dmesg	Displays the system messages on the console.
dos	DOS utilities.
dtype	Determines disk type.
du	Summarizes disk usage.
dumpdir	Prints the names of files on a backup archive.
echo	Echoes arguments.
ed	Invokes the text editor.
enable	Turns on terminals.
env	Sets environment for command execution.
ex	Invokes a text editor.
expr	Evaluates arguments as an expression.
factor	Factor a number.
false	Returns with a nonzero exit value.
file	Determines file type.
find	Finds files.
finger	Finds information about users.
format	Formats media
fsck	Checks and repairs file systems.
getopt	Parses command options.
grep, egrep, fgrep	Searches a file for a pattern.
grpcheck	Checks group file.
haltsys	Closes out the file systems and halts the CPU.
hd	Displays files in hexadecimal format.
hdinit	Creates a bootable XENIX system on a fixed disk.
head	Prints the first few lines of a stream.
hello	Write to another user.
help	Locates a manual page for a command.
id	Prints user and group IDs and names.
install	Install commands.
ipcrm	Removes a message queue, semaphore set, or shared memory ID.
ipcs	Reports the status of inter-process communication facilities.
join	Joins two relations.
keyboard	Activates international console keyboards.
kill	Terminates a process.
line	Reads one line.

ln	Makes a link to a file.
local	Allows printing on terminal printer.
login	Gives access to the system.
logname	Gets login name.
look	Finds lines in a sorted list.
lp, lpr, cancel	Sends/cancels requests to lineprinter.
lpadmin	Configures the lineprinter spooling system.
lpinit	Adds new lineprinters to system.
lpsched, lpshut, lpmove	Starts/stops the lineprinter request scheduler and moves requests.
lpstat	Prints lineprinter status information.
ls, ll, l, lc, lr, lx	Gives information about contents of directories.
mail	Sends, reads or disposes of mail.
makefs	Makes a file system.
msg	Permits or denies messages sent to a terminal.
mkdir	Makes a directory.
mkfs	Constructs a file system.
mknod	Builds special files.
mkuser	Adds a login ID to the system.
more	Views a file one full screen at a time.
mount	Mounts a file structure.
mv, mvdir	Moves or renames files/directories.
ncheck	Generates names from inode numbers.
netutil	Administers the XENIX network.
newform	Changes the format of a text file.
newgrp	Logs user in to a new group.
news	Prints news items.
nice	Runs a command at a different priority.
nl	Adds line numbers to a file.
nohup	Runs a command immune to hangups and quits.
od	Displays files in octal format.
pack, pcat, unpack	Compresses and expands files.
passwd	Changes login password.
pg	File perusal filter for soft-copy terminals.
pr	Prints files on the standard output.
ps	Reports process status.
pstat	Reports system information.
pwadmin	Performs password aging administration.
pwcheck	Checks password file.
pwd	Prints working directory name.
quot	Summarizes file system ownership.
random	Generates a random number.
rcp	Copies files across XENIX systems.
red	Invokes a restricted version of <i>ed</i> (C).
remote	Executes commands on a remote XENIX system.

restore, restor	Invokes incremental file system restorer.
rewind	Rewinds the tape cartridge system.
rm, rmdir	Removes files or directories.
rmuser	Removes a user from the system.
rsh	Invokes a restricted shell (command interpreter).
runbig	Runs a command that requires more memory than the default allows.
sddate	Prints and sets backup dates.
sdiff	Compares files side-by-side.
sed	Invokes the stream editor.
setclock	Sets/displays the real-time clock.
setcolor	Sets foreground and background colors on a color monitor.
setkey	Assigns the function keys.
setmnt	Establishes /etc/mnttab table.
settime	Changes the access and modification dates of files.
sh	Invokes the shell command interpreter.
shutdown	Terminates all processing.
skip	Skips volume(s) on the tape cartridge drive.
sleep	Suspends execution for an interval.
sort	Sorts and merges files.
split	Splits a file into pieces.
stty	Sets the options for a terminal.
su	Makes the user a super-user or another user.
sulogin	Accesses single user mode.
sum	Calculates checksum and counts blocks in a file.
sync	Updates the super-block.
sysadmin	Performs file system backups and restores files.
tail	Delivers the last part of a file.
tar	Archives files.
tars	Archives files to tape.
tdd	Buffers and copies a file.
tee	Creates a tee in a pipe.
test	Tests conditions.
touch	Updates access and modification times of a file.
tr	Translates characters.
true	Returns with a zero exit value.
tset	Sets terminal modes.
tty	Gets the terminal's name.
umask	Sets file-creation mode mask.
umount	Dismounts a file structure.
uname	Prints the current XENIX name.
uniq	Reports repeated lines in a file.
units	Converts units.
uuclean	Clean-up the uucp spool directory.

uucp, uulog, uuname	Copies files from XENIX to XENIX.
uuinstall	Administers uucp control files.
uustat	Uucp status inquiry and job control.
uusub	Monitor uucp network.
uuto, uupick	Public XENIX-to-XENIX file copy.
uux	Executes command on remote XENIX.
vi	Invokes a screen-oriented display editor.
vsh	Invokes the visual shell.
wait	Awaits completion of background processes.
wall	Writes to all users.
wc	Counts lines, words, and characters.
what	Identifies files.
who	Lists who is on the system.
whodo	Determines who is doing what.
write	Writes to another user.
xargs	Constructs and executes commands.
xmail	Improved mail program.
yes	Prints string repeatedly.



Name

intro – Introduces XENIX commands.

Description

This section describes use of the individual commands available in the XENIX Timesharing System. Unless otherwise noted, commands described in this section accept options and other arguments according to the following syntax:

name [*option(s)*] [*cmdarg(s)*]

where:

name Is the name of an executable file.

option *–noargletter(s)* or, *–argletter*◇*optarg* (where ◇ is optional whitespace).

noargletter Is a single letter representing an option without an argument.

argletter Is a single letter representing an option requiring an argument.

optarg Is an argument (character string) satisfying preceding *argletter*.

cmdarg Is a pathname (or other command argument) *not* beginning with *–*. By itself, *–* indicates the standard input.

See Also

getopt(C), getopt(S)

Diagnostics

Upon termination, each command returns two bytes of status, one supplied by the system and giving the cause for termination, and (in the case of “normal” termination) one supplied by the program (see *wait(S)* and *exit(S)*). The former byte is 0 for normal termination; the latter is customarily 0 for successful execution and nonzero to indicate troubles such as erroneous parameters, or bad or inaccessible data. It is called variously “exit code”, “exit status”, or “return code”, and is described only where special conventions are involved.

Notes

Not all commands adhere to the syntax described here.



Name

accept, reject – Allows/prevents print requests to a lineprinter or class of printers.

Syntax

```
/usr/lib/accept destinations  
/usr/lib/reject [ -r[ reason ] ] destinations
```

Description

Accept allows *lp*(C) to accept requests for the named *destinations*. A *destination* can be either a printer or a class of printers. Use *lpstat*(C) to find the status of *destinations*.

Reject prevents *lp*(C) from accepting requests for the named *destinations*. A *destination* can be either a printer or a class of printers. Use *lpstat*(C) to find the status of *destinations*. The following option is useful with *reject*:

-r[*reason*] Associates a *reason* with preventing *lp* from accepting requests. This *reason* applies to all printers mentioned up to the next **-r** option. *Reason* is reported by *lp* when users direct requests to the named *destinations*. *Reason* is also reported by *lpstat*(C). If *reason* is more than one word, you must enclose it within quotes. If the **-r** option is not present or the **-r** option is given without a *reason*, the default *reason* ("reason unknown") will be used.

Files

/usr/spool/lp/*

See Also

enable(C), lp(C), lpadmin(C), lpinit(C), lpsched(C), lpstat(C).



Name

acctcom – Searches for and prints process accounting files.

Syntax

```
acctcom [[ options ][ file ]] ...
```

Description

Acctcom reads *file*, the standard input, or */usr/adm/pacct*, in the form described by *acct(F)* and writes selected records to the standard output. Each record represents the execution of one process. The output shows the COMMAND NAME, USER, TTYNAME, START TIME, END TIME, REAL (SEC), CPU (SEC), MEAN SIZE (K), and optionally, F (the *fork/exec* flag: 1 for *fork* without *exec*) and STAT (the system exit status).

The command name is prepended with a # if during execution of the command, the user was given super-user privileges (i.e. set uid root). If a process is not associated with a known terminal, a ? is printed in the TTYNAME field.

If no *file* is specified, and if the standard input is associated with a terminal or */dev/null* (as is the case when using *&* in the shell), */usr/adm/pacct* is read, otherwise the standard input is read.

If any *file* arguments are given, they are read in their respective order. Each file is normally read forward, i.e., in chronological order by process completion time. The */usr/adm/pacct* file is usually the current file to be examined; a busy system may need several files, in which case all but the current file will be found in */usr/adm/pacct?*. The *options* are:

- b Reads backwards, showing latest commands first.
- f Prints the *fork/exec* flag and system exit status columns in the output.
- h Instead of mean memory size, shows the fraction of total available CPU time consumed by the process during its execution. This “hog factor” is computed as:

$$(\text{total CPU time})/(\text{elapsed time}).$$
- i Prints columns containing the I/O counts in the output.
- k Instead of memory size, shows total kcore-minutes.
- m Shows mean core size (the default).

- r** Shows CPU factor (user-time/(system-time + user-time).)
- t** Shows separate system and user CPU times.
- v** Excludes column headings from the output.
- l *line*** Shows only processes belonging to terminal */dev/line*.
- u *user*** Shows only processes belonging to *user* that may be specified by a user ID, a login name that is then converted to a user ID, a # which designates only those processes executed with super-user privileges, or ? which designates only those processes associated with unknown user IDs.
- g *group*** Shows only processes belonging to *group*. The *group* may be designated by either the group ID or group name.
- d *mzn/dd*** Any *time* arguments following this flag are assumed to occur on the given month and day, rather than during the last 24 hours. This is needed for looking at old files.
- s *time*** Shows only those processes that existed on or after *time*, given in the form *hr:min:sec*. The *:sec* or *:min:sec* may be omitted.
- e *time*** Shows only those processes that existed on or before *time*. Using the same *time* for both **-s** and **-e** shows the processes that existed at *time*.
- n *pattern*** Shows only commands matching *pattern* that may be a regular expression as in *ed(C)* except that + means one or more occurrences.
- H *factor*** Shows only processes that exceed *factor*, where *factor* is the “hog factor” as explained in option **-h** above.
- O *time*** Shows only those processes with operating system CPU time that exceeds *time*.
- C *time*** Shows only those processes that exceed *time* (the total CPU time).

Multiple options have the effect of a logical AND.

Files

/etc/passwd
 /usr/adm/pacct
 /etc/group

See Also

acct(F) acct(S), accton(C), ps(C), su(C), utmp(F)

Notes

Acctcom reports only on processes that have terminated; use *ps*(C) for active processes.



Name

accton – Turns on accounting.

Syntax

accton [*file*]

Description

Accton turns on and off process accounting. If no *file* is given, then accounting is turned off. If *file* is given, it must be the name of an existing file, to which the kernel appends process accounting records. (see *acct*(S) and *acct*(F)).

Files

/etc/passwd	Used for login name to user ID conversions
/usr/lib/acct	Holds many accounting commands
/usr/adm/pacct	Current process accounting file
/usr/adm/wtmp	Login/logout history file

See Also

acct(F), acct(S), acctcom(C), utmp(F)



Name

asktime – Sets/displays the correct time of day.

Syntax

/etc/asktime

Description

This command prompts for the time of day and sets the system clock to the time you enter. You must enter a legal time according to the proper format as defined below:

[yymmdd]hhmm

Here the first *mm* is the month number; *dd* is the day number in the month; *hh* is the hour number (24-hour system); the second *mm* is the minute number; *yy* is the last two digits of the year number and is optional. The current year is the default if no year is mentioned.

Examples

This example sets the new time, date, and year to “11:29 April 20, 1984”.

Current System Time is Wed Nov 3 14:36:23 PST 1985

Enter time (*[yymmdd]hhmm*): 8404201129

Diagnostics

If you enter an illegal time, *asktime* prompts with:

cvdate: bad conversion

Try again:

Notes

The command is privileged, and can be executed only by the super-user.



Name

assign, deassign – Assigns and deassigns devices.

Syntax

assign [**-u**] [**-v**] [**-d**] [*device*] ...

deassign [**-u**] [**-v**] [*device*] ...

Description

Assign attempts to assign *device* to the current user. The *device* argument must be an assignable device that is not currently assigned. An *assign* command without an argument prints a list of assignable devices along with the name of the user to whom they are assigned.

Deassign is used to “deassign” devices. Without any arguments, *deassign* will deassign all devices assigned to the user. When arguments are given, an attempt is made to deassign each *device* given as an argument.

Available options include:

- d** Performs the action of *deassign*. The **-d** option causes all devices whose names appear after the option to be deassigned.
- v** Gives verbose output.
- u** Suppresses assignment or deassignment, but performs error checking.

The *assign* command will not assign any assignable devices if it cannot assign all of them. *Deassign* gives no diagnostic if the *device* cannot be deassigned. Devices may be automatically deassigned at logout, but this is not guaranteed. *Device* names may be just the beginning of the device required. For example,

```
assign fd
```

should be used to assign all floppy disk devices. Raw versions of *device* will also be assigned, e.g., the raw floppy disk devices */dev/rfd?* would be assigned in the above example.

Note that in many installations the assignable devices such as floppy disks have general read and write access, so the *assign* command may not be necessary. This is particularly true on one-user systems. Devices supposed to be assignable with this command should be owned by the user *asg*. The directory */dev* should be owned by *bin* and have mode 755. The *assign* command (after checking for use by someone else) will then make the device owned by whoever invokes the command, without changing the access permissions. This allows the system administrator to

set up individual devices that are freely available, assignable (owned by *asg*), or nonassignable and restricted (not owned by *asg* and with some restricted mode).

Note also that the first time *assign* is invoked it builds the assignable devices table */etc/atab*. This table is used in subsequent invocations to save repeated searches of the */dev* directory. If one of the device entries has changed (i.e., owned by *asg* or no longer owned by *asg*), */etc/atab* should be removed (by the super-user) so that a correct list will be built the next time the command is invoked.

Files

/etc/atab Table of assignable devices

/dev/asglock File to prevent concurrent access

Diagnostics

Exit code 0 returned if successful, 1 if problems, 2 if *device* cannot be assigned.

Name

at, *batch* – Executes commands at a later time.

Syntax

at time [date] [+ increment]

at -r job ...

at -l [job ...]

Description

At and *batch* read commands from standard input to be executed at a later time. *At* allows you to specify a time when the commands should be executed, while *batch* executes jobs when the system load level permits.

Standard output and standard error output are mailed to the user unless they are redirected elsewhere. The shell environment variables, current directory, *umask*, and *ulimit* are retained when the commands are executed. Open file descriptors, traps, and priority are lost.

Users are permitted to use *at* if their name appears in the file, */usr/lib/cron/at.allow*. If that file does not exist, the */usr/lib/cron/at.deny* file is checked to determine if the user should be denied access to *at*. If neither file exists, only the super-user is allowed to submit a job. If only the *at.deny* file exists, global usage is permitted. The *allow/deny* files consist of one user name per line.

The options are:

time The *time* may be specified as one, two, or four digits. One and two digit numbers are taken to be hours, four digits to be hours and minutes. The time may alternately be specified as two numbers separated by a colon, meaning *hour:minute*. A suffix *am* or *pm* may be appended; otherwise a 24-hour clock time is understood. The suffix *zulu* may be used to indicate GMT. The special names *noon*, *midnight*, *now*, and *next* are also recognized.

date An optional *date* may be specified as either a month name followed by a day number (and possibly year number preceded by an optional comma) or a day of the week (fully spelled, or abbreviated to three characters). Two special "days," *today* and *tomorrow*, are recognized. If no *date* is given, *today* is assumed if the given hour, is greater than the current hour, and *tomorrow* is assumed if it is less. If the given month is less than the current month (and no year is given), the next year is assumed.

increment

The optional *increment* is simply a number suffixed by one of the following: **minutes, hours, days, weeks, months, or years**. (The singular form is also accepted.) Thus, legitimate commands include:

```
at 0815am Jan 24
at 8:15am Jan 24
at now + 1 day
at 5 pm Friday
```

-r

Removes jobs previously scheduled by the *at* or *batch* command. You can only remove your own jobs, unless you are the super-user.

-l

Lists all the jobs currently scheduled for the invoking user.

At and *batch* write the job number and schedule time to standard error. *Batch* submits a batch job. It is almost equivalent to “at now”, but not quite: *batch* goes into a different queue; *at now* will respond with the error message too late.

Examples

The *at* and *batch* commands read from standard input the commands to be executed at a later time. *Sh*(C) provides different ways of specifying standard input. Within your commands, it may be useful to redirect standard output.

This sequence can be used at a terminal:

```
batch
nroff filename > outfile
<CTRL-D> (hold down “CTRL” and press “D”)
```

This sequence, which demonstrates redirecting standard error to a pipe (|), is useful in a shell procedure (the sequence of output redirection specifications is significant):

```
batch <<!
nroff filename 2>&1 >outfile | mail loginid
!
```

To have a job reschedule itself, invoke *at* from within the shell procedure, by including code similar to the following within the shell file:

```
echo “sh shellfile” | at 1900 thursday next week
```

Files

<code>/usr/lib/cron</code>	Main cron directory
<code>/usr/lib/cron/at.allow</code>	List of allowed users
<code>/usr/lib/cron/at.deny</code>	List of denied users
<code>/usr/lib/cron/queue</code>	Scheduling information
<code>/usr/spool/cron/atjobs</code>	Spool area

See Also

`cron(C)`, `kill(C)`, `mail(C)`, `nice(C)`, `ps(C)`, `sh(C)`

Diagnostics

Complains about syntax errors and times that are out of range.



Name

awk – Searches for and processes a pattern in a file.

Syntax

```
awk [ -Fc ] [ -f programfile | 'program' ] [ parameters ] [ files ]
```

Description

Awk scans each input *file* for lines that match patterns specified in *program* or in *programfile*. When a line of *files* matches a pattern, an associated action may be performed. *Awk* is useful for compiling information, performing arithmetic on input data, and for doing iterative or conditional processing.

The options are:

- Fc** Sets the field separator variable (FS) to the letter ‘c’. The default field separators are TAB and SPACE.
- f** Causes *awk* to take its program from *programfile*.

The arguments are:

- programfile* A file containing an *awk* program.
- program* An *awk* program. Programs given on the command line must be enclosed in single quotation marks to prevent interpretation by the shell.
- parameters* May be passed to *awk*, in the form *x=...*, *y=...*, etc.
- files* The name(s) of the file or files to be processed. If no filename is given, the standard input is used.

An *awk* program consists of statements in the form:

```
pattern { action }
```

Patterns are always enclosed by slash characters. For example, the pattern `/[AEIOUaeiou]/` matches any input line beginning with a vowel.

Action statements are always enclosed by left and right braces { }.

Pattern-action statements may appear on the *awk* command line or in an *awk* program file.

If no *pattern* is given, all lines in the input file are matched. If no *action* is given, each matched line is displayed on the standard output.

A *pattern* may be a literal string or a regular expression, or a combination of a regular expression and a field or variable separated by operators.

Awk also provides two patterns, BEGIN and END, that can be used to perform actions before the first line is read and after the last line is read, respectively. If used, BEGIN and END must be the first and last patterns in the program.

To select a range of lines, use two patterns on a single program line, separated by a comma.

An action is a sequence of statements. Each action is separated by a semicolon, NEWLINE, or right brace. Multiple actions may occur on the same program line separated with a semicolon. The last action on the program line must terminate in a semicolon, unless it is followed by a right brace. See *Statements* later in this section.

Variables

In addition to variables declared and initialized by the user, *awk* has the following program variables:

NR	Number of records.
NF	Number of fields in a record.
FS	Input field separator.
OFS	Output field separator.
RS	Input record separator.
ORS	Output record separator.
\$0	The current record.
\$1, \$n	Fields in the current record.
OFM	The output format for numbers. The default is %.6g.
FILENAME	The name of the input file currently being read.

Arrays may be used to store data. Arrays do not need to be dimensioned before use. For example, “w[i]” denotes the *i*th item of array *w*.

Expressions

A pattern match with a field or variable may be tested with the following operators:

- `~` Matches the regular expression.
- `!~` Does not match the regular expression.

Awk processes relational expressions using the following operators:

- `<` Less than
- `<=` Less than or equal to
- `==` Equal to
- `!=` Not equal to
- `>=` Greater than or equal to
- `>` Greater than

Patterns can be combined using the following operators:

- `&&` And
- `||` Or
- `!` Not

An empty expression-list stands for the whole line. Expressions take on string or numeric values as appropriate, and are built using the following operators:

- `+` Addition
- `-` Subtraction
- `*` Multiplication
- `/` Division
- `%` Modulo

Concatenation is indicated by a blank.

The following C operators are also available for expressions:

- `++` Increment

--	Decrement
+=	Add and assign
-=	Subtract and assign
*=	Multiply and assign
/=	Divide and assign
%=	Modulo and assign

Statements

```

if ( conditional ) statement [ else statement ]
while ( conditional ) statement
for ( expression ; conditional ; expression ) statement
break
continue
{ [ statement ] ... }
variable = expression
print [ expression-list ] [ >expression ]
printf format [ , expression-list ] [ >expression ]
next      #skip remaining patterns on input line

```

while	Used the same as in C.
for	The iterative construction. It can be used the same as in the C language, or as an array iterator.
break	Breaks out of a <i>while</i> or <i>for</i> loop. Similar to its C counterpart.
continue	Continues processing at the next iteration of a <i>while</i> or <i>for</i> loop. Similar to its C counterpart.
print	Prints its arguments on the standard output, or in a file if redirected.
printf	Prints <i>expression-list</i> in the format specified in <i>format</i> . See <i>printf(S)</i> .
next	Stops processing the current record and moves to the next record, if any.

Comments are preceded by a number sign (#).

Functions

Awk has the following built-in functions:

exit(<i>x</i>)	Terminates the <i>awk</i> program. If <i>x</i> is given, this value is <i>awk</i> 's return value. If <i>x</i> is not given, 0 is returned. If the program has an END section, it is invoked before termination.
exp(<i>x</i>)	Exponentiation of the value of <i>x</i> .
index(<i>s</i>, <i>t</i>)	Returns the starting position of the leftmost occurrence of <i>t</i> in <i>s</i> . If <i>t</i> is not a substring of <i>s</i> , then <i>index(s, t)</i> is 0.
int(<i>x</i>)	Returns the largest integer less than or equal to <i>x</i> . If <i>x</i> is negative, its value is the smallest integer greater than or equal to <i>x</i> .
length(<i>x</i>)	A function whose value is the number of characters in the string (<i>x</i>). With no arguments, <i>length</i> is equivalent to \$0.
log(<i>x</i>)	Natural logarithm of <i>x</i> .
split(<i>x</i>, <i>y</i>)	Assigns the fields of string <i>x</i> to successive elements of array <i>y</i> .
sqrt(<i>x</i>)	Square root of <i>x</i> .
substr(<i>string</i>, <i>index</i>, <i>length</i>)	Returns the substring of <i>string</i> that begins at <i>index</i> and is <i>length</i> characters long.
sprintf(format [, <i>expression-list</i>])	Formats the expressions according to <i>format</i> and returns the resulting string.

Examples

The following displays lines in *file* longer than 72 characters:

```
awk ' {length > 72} ' file
```

The following prints the first two fields in opposite order:

```
awk ' { print $2, $1 } ' file
```

The following *awk* program adds up the first columns and prints their sum and average:

```
      { s += $1 }
END    { print "sum is", s, " average is", s/NR }
```

The following prints the fields in *file* in reverse order:

```
awk '{ for (i = NF; i > 0; --i) print $i }' file
```

The following prints all lines between start/stop pairs:

```
awk '/start/, /stop/' file
```

The following *awk* program file will print all lines in the object file whose first field is different from the first field in the previous line:

```
$1 != prev { print; prev = $1 }
```

The following program prints a file, filling in page numbers starting at 5:

```
/Page/ {$2 = n++;}  
{print $1 $2}
```

The command line has the form: `awk -f program n=5 input`

See Also

grep(C), lex(CP), malloc(S), sed(C)

XENIX Text Processing Guide

Notes

Input white space is not preserved on output if fields are involved.

There are no explicit conversions between numbers and strings. To force an expression to be treated as a number, add 0 to it; to force it to be treated as a string, concatenate the null string ("") to it.

This command is more fully documented in the *XENIX Text Processing Guide*.

Name

backup, dump – Performs incremental file system backup.

Syntax

backup [*key* [*arguments*] *filesystem*]

dump [*key* [*arguments*] *filesystem*]

Description

Backup copies to the specified device all files changed after a certain date in the *filesystem*. *Backup* and *dump* are equivalent commands and may be used interchangeably. The *key* specifies the date and other options about the backup, where a *key* consists of characters from the set **0123456789kfusd**. The meanings of these characters are described below:

- f** Places the backup on the next *argument* file instead of the default device.
- u** If the backup completes successfully, writes the date of the beginning of the backup to the */etc/ddate* file. This file records a separate date for each file system and each backup level.
- 0–9** This number is the “backup level.” This option backs up all files modified since the last date stored in the */etc/ddate* file for the same file system at lesser levels. If no date is determined by the level, the beginning of time is assumed; thus the option **0** causes the entire file system to be backed up.
- s** For backups to magnetic tape, the size of the tape specified in feet. The number of feet is taken from the next *argument*. When the specified size is reached, *backup* will wait for reels to be changed. The default size is 2,300 feet.
- d** For backups to magnetic tape, the density of the tape, expressed in BPI, is taken from the next *argument*. This is used in calculating the amount of tape used per write. The default is 1600.
- k** This option is used when backing up to a block-structured device, such as a floppy disk. The size (in K bytes) of the volume being written is taken from the next *argument*. The number of Kbytes for a high-capacity diskette is 1200 and for a low-capacity diskette is 360. If the **k** argument is specified, any **s** and **d** arguments are ignored. The default is to use **s** and **d**.

If no arguments are given, the *key* is assumed to be **9u** and a default file system is backed up to the default device.

The default file system and the backup device depend on the settings of the variables DISK and TAPE, respectively, in the `/etc/default/backup` file.

A convenient way to make different levels of backups is provided through the `sysadmin(C)` command. If `backup` is used directly, the following backup strategy is recommended.

The first backup should be a full level-0 backup:

```
backup 0u
```

Next, periodic level-9 backups should be made:

```
backup 9u
```

When the level-9 incremental backup becomes unmanageable because a tape is full or too many floppies are required, a level-1 backup should be made:

```
backup 1u
```

Further periodic level-9 backups are based on the level-1 backup, which is based on the level-0 full backup. This progression of levels of backups can be carried as far as desired.

Files

<code>/etc/ddate</code>	Records backup dates of file system/level
<code>/etc/default/backup</code>	Default backup information

See Also

`backup(F)`, `cpio(C)`, `default(M)`, `dumpdir(C)`, `restore(C)`, `sysadmin(C)`

XENIX Operations Guide

Diagnostics

If the backup requires more than one volume (where a volume is likely to be a floppy disk or tape), you will be asked to change volumes. Press RETURN after changing volumes.

Notes

Sizes are based on 1600 BPI for blocked tape; the raw magnetic tape device has to be used to approach these densities. Write errors to the backup device are usually fatal. Read errors on the file system are ignored.

Warning

When backing up to floppy disks, be sure to have enough *formatted* floppies ready before starting a backup.



Name

banner – Prints large letters.

Syntax

banner *strings*

Description

Banner prints its arguments (each up to ten characters long) in large letters on the standard output. This is useful for printing names at the front of printouts.

See Also

echo(C)



Name

basename – Removes directory names from pathnames.

Syntax

basename *string* [*suffix*]

Description

Basename deletes any prefix ending in / and the *suffix* (if present in *string*) from *string*, and prints the result on the standard output. The result is the “base” name of the file, i.e., the filename without any preceding directory path and without an extension. It is used inside substitution marks (“”) in shell procedures to construct new filenames. The *suffix* may be a regular expression (see *ed*(C)) matching the end of *string*.

The related command, *dirname*, deletes the last level from *string* and prints the resulting path on the standard output.

Examples

The following command displays the **memos** filename on the standard output:

```
basename /usr/johnh/memos.old .old
```

The following shell procedure, when invoked with the `/usr/src/cmd/cat.c` argument, compiles the named file and moves the output to a file named **cat** in the current directory:

```
cc $1
mv a.out `basename $1 .c`
```

Note: this example assumes the file name given will end with **.c**. Because a period matches any character in a regular expression, *basename* will remove the last two characters if the last character is a **c**.

See Also

dirname(C), *sh*(C)



Name

`bc` – Invokes a calculator.

Syntax

`bc [-c] [-l] [file ...]`

Description

Bc is an interactive processor for a language that resembles C but provides unlimited precision arithmetic. It takes input from any files given, then reads the standard input. The `-l` argument stands for the name of an arbitrary precision math library. The syntax for *bc* programs is as follows: L means the letters “a–z”, E means “expression”, S means “statement”.

Comments:

Enclosed in `/*` and `*/`

Names:

Simple variables: L

Array elements: L [E]

The words “ibase”, “obase”, and “scale”

Other operands:

Arbitrarily long numbers with optional sign and decimal point
(E)

`sqrt` (E)

`length` (E) Number of significant decimal digits

`scale` (E) Number of digits right of decimal point

`L` (E , ... , E)

Additive operators:

`+`

`-`

Multiplicative operators:

`*`

`/`

`%` (remainder)

`^` (exponentiation)

Unary operators:

++
-- (prefix and suffix; apply to names)

Relational operators:

==
<=
>=
!=
<
>

Assignment operators:

=
=+
=-
=*
=/
=%
=^

Statements:

E
{ S ; ... ; S }
if (E) S
while (E) S
for (E ; E ; E) S
null statement
break
quit

Function definitions:

```
define L ( L ,..., L ) {
    auto L, ... , L
    S ; ... S
    return ( E )
}
```

Functions in -l math library:

s(x)	Sine
c(x)	Cosine
e(x)	Exponential
l(x)	Log
a(x)	Arctangent
j(n,x)	Bessel function

All function arguments are passed by value.

The value of a statement that is an expression is printed unless the main operator is an assignment. Either semicolons or NEWLINES may separate statements. Assignment to *scale* influences the number of digits to be retained on arithmetic operations in the manner of *dc*(C). Assignments to *ibase* or *obase* set the input and output radix numbers, respectively.

The same letter may be used simultaneously as an array, a function, and a simple variable. All variables are global to the program. "Auto" variables are pushed down during function calls. When using arrays as function arguments or defining them as automatic variables, empty square brackets must follow the array name.

Bc is actually a preprocessor for *dc*(C), which it invokes automatically, unless the *-c* (compile only) option is present. If the *-c* option is present, the *dc* input is sent to the standard output instead.

Example

The following function computes an approximate value of the exponential function:

```
scale = 20
define e(x){
    auto a, b, c, i, s
    a = 1
    b = 1
    s = 1
    for(i=1; 1==1; i++){
        a = a*x
        b = b*i
        c = a/b
        if(c == 0) return(s)
        s = s+c
    }
}
```

The following prints the approximate values of the exponential function of the first ten integers:

```
for(i=1; i<=10; i++) e(i)
```

Files

/usr/lib/lib.bc	Mathematical library
/usr/bin/dc	Desk calculator proper

See Also

dc(C)

The XENIX User's Guide

Notes

A *For* statement must have all three E's.

Quit is interpreted when read, not when executed.

Name

`bdiff` – Compares files too large for *diff*.

Syntax

`bdiff file1 file2 [ n ] [-s]`

Description

Bdiff compares two files, finds lines that are different, and prints them on the standard output. It allows processing of files that are too large for *diff*. *Bdiff* splits each file into *n*-line segments, beginning with the first nonmatching lines, and invokes *diff* upon the corresponding segments. The arguments are:

- n* The number of lines which *bdiff* splits each file into for processing. The default value is 3500. This is useful when 3500-line segments are too large for *diff*.
- `-s` Suppresses printing of *bdiff* diagnostics. Note that this does not suppress printing of diagnostics from *diff*.

If *file1* (or *file2*) is a dash (`-`), the standard input is read.

The output of *bdiff* is exactly that of *diff*. Line numbers are adjusted to account for the segmenting of the files, and the output looks as if the files had been processed whole.

Files

`/tmp/bd????`

See Also

`diff(C)`

Notes

Because of the segmenting of the files, *bdiff* does not necessarily find the smallest sufficient set of file differences.



Name

bfs – Scans big files.

Syntax

bfs [-] *name*

Description

Bfs is like *ed*(C) except that it is read-only and processes much larger files. Files can be up to 1024K bytes with 32K lines, and up to 255 characters per line. *Bfs* is usually more efficient than *ed* for scanning a file, since the file is not copied to a buffer. It is most useful for identifying sections of a large file where *csplic*(C) can be used to divide it into more manageable pieces for editing.

Normally, the size of the file being scanned is printed, as is the size of any file written with the *w* command. The optional dash (-) suppresses printing of sizes. The input prompt is initially a blank. Type a ‘P’ and press RETURN if you wish a visible prompt of an asterisk (*). The ‘P’ acts as a toggle, so prompting can be turned off again by entering another ‘P’ and a RETURN. Note that messages are given in response to errors only if prompting is turned on.

All address expressions described under *ed* are supported. In addition, regular expressions may be surrounded with two symbols other than the standard slash (/) and (?): A greater-than sign (>) indicates downward search without wraparound, and a less-than sign (<) indicates upward search without wraparound. Since *bfs* uses a different regular expression-matching routine from *ed*, the regular expressions accepted are slightly wider in scope (see *regex*(S)). There is also a slight difference in mark names: only the letters ‘a’ through ‘z’ may be used, and all 26 marks are remembered.

The *e*, *g*, *v*, *k*, *n*, *p*, *q*, *w*, *=*, *!*, and null commands operate as described under *ed*. Commands such as *--*, *++*, *+=*, *+++=*, *-l2*, and *+4p* are accepted. Note that *l,l0p* and *l,l0* will both print the first ten lines. The *f* command only prints the name of the file being scanned; there is no remembered filename. The *w* command is independent of output diversion, truncation, or crunching (see the *xo*, *xt*, and *xc* commands, below). The following additional commands are available:

xf *file*

Further commands are taken from the named *file*. When an end-of-file is reached, an interrupt signal is received, or an error occurs, reading resumes with the file containing the *xf*. *Xf* commands may be nested to a depth of 10.

xo [*file*]

Further output from the *p* and null commands is diverted to the named *file*. If *file* is missing, output is diverted to the standard output. Note that each diversion causes either truncation or creation of the file.

: *label*

This positions a *label* in a command file. The *label* is terminated by a NEW-LINE; any spaces between the **:** and the start of the *label* are ignored. This command may also be used to insert comments into a command file, since labels need not be referenced.

(.,.)*xb/regular expression/label*

A jump (either upward or downward) is made to *label* if the command succeeds. It fails under any of the following conditions:

1. Either address is not between 1 and \$.
2. The second address is less than the first.
3. The regular expression doesn't match at least one line in the specified range, including the first and last lines.

If successful, the command sets dot (.) to the line matched and a jump is made to *label*. This command is the only one that doesn't issue an error message on bad addresses, so it may be used to test whether addresses are bad before other commands are executed. Note that the command:

xb/^/label

is an unconditional jump.

The *xb* command is allowed only if it is read from somewhere other than a terminal. If it is read from a pipe, only a downward jump is possible.

xt *number*

Output from the *p* and null commands is truncated to a maximum of *number* characters. The initial number is 255.

xv[*digit*][*spaces*][*value*]

The variable name is the specified *digit* following the *xv*. *Xv5100* or *xv5 100* both assign the value 100 to the variable 5. *Xv61,100p* assigns the value 1,100p to the variable 6. To reference a variable, put a % in front of the variable name. For example, you can use these assignments for variables 5 and 6:

```
1,%5p
1,%5
%6
```

prints the first 100 lines.

g/%5/p

globally searches for the characters *100* and prints each line containing a match. To escape the special meaning of %, a \ must precede it. For example:

```
g/".*\%[cds]/p
```

could be used to match and list lines containing *printf* characters, decimal integers, or strings.

Another feature of the *xv* command is that the first line of output from a XENIX command can be stored into a variable. The only requirement is that the first character of *value* be a !. For example:

```
xv5!cat junk
!rm junk
!echo "%5"
xv6!expr %6 + 1
```

puts the current line in variable 5, prints it, and increments variable 5 by one. To escape the special meaning of ! as the first character of *value*, precede it with a \. For example:

```
xv7!\!date
```

stores the *!date* value in variable 7.

xbz label

xbn label

These two commands test the last saved *return code* from the execution of a XENIX command (*!command*) for zero or nonzero value, respectively, and jump to the specified label. The two following examples search for the next five lines containing the string *size*:

```
xv55
:1
/size/
xv5!expr %5 - 1
!if 0%5 != 0 exit 2
xbn 1
```

```
xv45
:1
/size/
xv4!expr %4 - 1
!if 0%4 = 0 exit 2
xbz 1
```

xc [*switch*]

If *switch* is 1, output from the *p* and null commands is crunched; if *switch* is 0, it isn't. Without an argument, *xc* reverses *switch*. Initially, *switch* is set for no crunching. Crunched output has strings of TABs and spaces reduced to one space with blank lines suppressed.

See Also

csplit(C), ed(C), regex(S)

Diagnostics

If prompting is turned off, a question mark is printed (?) for errors in commands. When prompting is on, self-explanatory error messages appear.

Name

cal – Prints a calendar.

Syntax

cal [[*month*] *year*]

Description

Cal prints a calendar for the specified *year*. If a *month* is also specified, a calendar for that month only is printed. If no arguments are specified, the current, previous, and following months are printed, along with the current date and time. The *year* must be a number between 1 and 9999; *month* must be a number between 1 and 12 or it must have enough characters to specify a particular month. For example, *May* must be given to distinguish it from March, but *S* is sufficient to specify September. If only a *month* string is given, only that month of the current year is printed.

Notes

Beware that “cal 84” refers to the year 84, not 1984.

The calendar produced is that for England and her colonies. Note that England switched from the Julian to the Gregorian calendar in September of 1752, at which time eleven days were excised from the year. To see the result of this switch, try “cal 9 1752”.



Name

`calendar` – Invokes a reminder service.

Syntax

`calendar [-]`

Description

Calendar consults the `calendar` file in the user's current directory and mails him lines that contain today's or tomorrow's date. Most reasonable month-day dates, such as "Sep. 7," "september 7", and "9/7", are recognized, but not "7 September", "7/9" or "07/09".

On weekends, "tomorrow" extends through Monday. Lines that contain the date of a Monday will be sent to the user on the previous Friday. This is not true for holidays.

When an argument is present, *calendar* does its job for every user who has a `calendar` file in his login directory and sends the user the results by *mail*(C). Normally this is done daily, in the early morning, under the control of *cron*(C).

Files

`calendar`

`/usr/lib/calprog` To figure out today's and tomorrow's dates

`/etc/passwd`

`/tmp/cal*`

`/usr/spool/cron/crontabs/root`

See Also

`cron`(C), `mail`(C)

Notes

To get reminder service, a user's `calendar` file must have read permission for all.

The global substitution character (*) maps to all valid values of the date field. For example, a line in the **calendar** file such as:

*/1 This is the first day of the month

sends mail on the first day of every month.



Name

`cat` – Concatenates and displays files.

Syntax

```
cat [ -u ] [ -s ] [ -v ] [ -t ] [ -e ] file ...
```

Description

Cat reads each *file* in sequence and writes it on the standard output. If no input file is given, or if a single dash (-) is given, *cat* reads from the standard input. The options are:

- `-s` Suppresses warnings about nonexistent files.
- `-u` Causes the output to be unbuffered.
- `-v` Causes non-printing characters (with the exception of TABs, NEWLINES, and FORMFEEDs) to be printed visibly. Control characters are printed as “^X” (CTRL-X); the DELETE character (octal 0177) is printed as “?”. Non-ASCII characters (with the high bit set) are printed as “M-x,” where *x* is the character specified by the seven low order bits.
- `-t` Causes a tab to be printed as “^T”. This option is ignored if the `-v` option is not specified.
- `-e` Causes a “\$” character to be printed at the end of each line (prior to the NEWLINE). This option is ignored if the `-v` option is not set.

No input file may have the same name as the output file unless it is a special file.

Examples

The following example displays *file* on the standard output:

```
cat file
```

The following example concatenates *file1* and *file2* and places the result in *file3*:

```
cat file1 file2 >file3
```

The following example concatenates *file1* and appends it to *file2*:

```
cat file1 >>file2
```

See Also

cp(C), pr(C)

Warning

Command lines such as:

```
cat file1 file2 > file1
```

will cause the original data in *file1* to be lost; therefore, you must be careful when using special shell characters.

Name

`cd` – Changes working directory.

Syntax

`cd [ directory ]`

Description

Cd changes the current directory to the directory specified by *directory*. If no *directory* is specified, the value of the shell parameter, `$HOME`, is used, and the user is placed in his home directory. The “`..`” argument moves up from a directory to the parent directory.

The user must have search (execute) permission in all directories specified in *directory*.

Examples

The following example changes the current directory to the user’s home directory:

```
cd
```

The following command changes the current directory to `/usr/joe/memos/meetings`:

```
cd /usr/joe/memos/meetings
```

The following command changes the current directory from `/usr/joe/memos/meetings` to `/usr/joe/accounts/overdue`:

```
cd ../../accounts/overdue
```

See Also

`chdir(S)`, `chroot(C)`, `pwd(C)`, `sh(C)`



Name

chgrp – Changes group ID.

Syntax

chgrp *group file ...*

Description

Chgrp changes the group ID of each *file* to *group*. The group may be either a decimal group ID or a group name found in the */etc/group* file.

Files

/etc/passwd

/etc/group

See Also

chown(C), chown(S), group(M), passwd(M)

Notes

Only the owner or the super-user can change the group ID of a file.



Name

`chmod` – Changes the access permissions of a file or directory.

Syntax

`chmod mode file ...`

Description

The *chmod* command changes the access permissions (or *mode*) of a specified file or directory. It is used to control file and directory access by users other than the owner and super-user. The *mode* may be an expression composed of letters and operators (called *symbolic mode*), or a number (called *absolute mode*).

A *chmod* command using *symbolic mode* has the form:

`chmod [who] += [permission ...] filename`

Who is one or any combination of the following letters:

- a** Stands for “all users”. If *who* is not indicated on the command line, **a** is the default. The definition of “all users” depends on the user’s *umask*. See *umask*(C).
- g** Stands for “group”, all users who have the same group ID as the owner of the file or directory.
- o** Stands for “others”, all users on the system.
- u** Stands for “user”, the owner of the file or directory.

The operators are:

+ Adds permission

– Removes permission

= Assigns the indicated permission and removes all other permissions (if any) for that *who*. If no permission is assigned, existing permissions are removed.

Permissions can be any combination of the following letters:

x Execute (search permission for directories)

r Read

w Write

s Sets owner or group ID on execution of the file to that of the owner of the file. The mode “u+s” sets the user ID bit for the file. The mode “g+s” sets the group ID bit. Other combinations have no effect.

t Saves text in memory upon execution. (“Sticky bit”, see *chmod(S)*). Only the mode “u+t” sets the sticky bit. All other combinations have no effect. This mode can only be set by the super-user.

Multiple symbolic modes may be given, separated by commas, on a single command line. See the following “Examples” section for sample permission settings.

A *chmod* command using *absolute mode* has the form:

```
chmod mode filename
```

where *mode* is an octal number constructed by performing a logical OR on the following:

4000	Set user ID on execution
2000	Set group ID on execution
1000	Sets the sticky bit (see <i>chmod(S)</i>)
0400	Read by owner
0200	Write by owner
0100	Execute (search in directory) by owner
0040	Read by group
0020	Write by group
0010	Execute (search in directory) by group
0004	Read by others
0002	Write by others
0001	Execute (search in directory) by others
0000	No permissions

Examples

Symbolic Mode

The following command gives all users execute permission for *file*:

```
chmod +x file
```

The following command removes read and write permission for group and others from *file*:

```
chmod go-rw file
```

The following command gives other users read and write permission for *file*:

```
chmod o+rw file
```

The following command gives read permission to group and other:

```
chmod g+r,o+r file
```

Absolute Mode

The following command gives all users read, write and execute permission for *file*:

```
chmod 0777 file
```

The following command gives read and write permission to all users for *file*:

```
chmod 0666 file
```

The following command gives read and write permission to the owner of *file* only:

```
chmod 0600 file
```

See Also

chmod(S), ls(C)

Notes

The user ID, group ID and sticky bit settings are only useful for binary executable files. They have no effect on shell scripts.



Name

chown – Changes owner ID.

Syntax

chown *owner file ...*

Description

Chown changes the owner ID of the *files* to *owner*. *Owner* may be either a decimal user ID or a login name found in the */etc/passwd* file.

Files

/etc/passwd

/etc/group

See Also

chgrp(C), chown(S), group(M), passwd(M)

Notes

Only the owner or the super-user can change a file's owner or group ID.



Name

chroot – Changes root directory for command.

Syntax

chroot *newroot command*

Description

The **chroot** command is executed relative to the new root. The meaning of any initial slashes (/) in pathnames is changed for a command and any of its children to *newroot*. Furthermore, the initial working directory is *newroot*.

Notice that:

chroot *newroot command* >**x**

creates the file, **x**, relative to the original root, not the new root.

This command is restricted to the super-user.

The new root pathname is always relative to the current root even if a *chroot* is currently in effect. The *newroot* argument is relative to the current root of the running process. Note that it is not possible to change directories to what was formerly the parent of the new root directory; i.e., the *chroot* command supports the new root as an absolute root for the duration of the *command*. This means that “/.” is always equivalent to “/”.

See Also

chdir(S)

Notes

Exercise extreme caution when referencing special files in the new root file system.



Name

`chsh` - Changes the default shell.

Syntax

`chsh user shell`

Description

The `chsh` command lets you change a *user's* default shell. You should use this method when changing the default shell as `chsh` handles all necessary file updates and assures that the desired shell is available.

Shell can be:

<code>sh</code>	Bourne shell
<code>csh</code>	C-shell
<code>vsh</code>	Visual shell

After you select the shell, `chsh` insures that the shell is installed on your system, then checks that all appropriate shell files are placed in your home directory. If you select the Bourne shell, `chsh` creates a `.profile` file. If you select the C-shell, `chsh` creates a `.login` file and a `.cshrc` file.

Files

`/usr/lib/mkuser/mkuser.prof`
`/usr/lib/mkuser/mkuser.login`
`/usr/lib/mkuser/mkuser.cshrc`
`/etc/passwd`

See Also

`passwd(C)`, `pwadmin(C)`



Name

`clear, cls` – Clears the terminal screen

 Syntax

`clear`
`cls`

Description

Clear clears your screen if it is possible. *Clear* looks in the environment for the terminal type and then in */etc/termcap* to figure out how to clear the screen.

Files

/etc/termcap

Name

clri – Clears inode.

Syntax

/etc/clri file-system i-number ...

Description

Clri writes zeros on the 64 bytes occupied by the inode numbered *i-number*. *File-system* must be a special filename referring to a device containing a file system. After *clri* is executed, any blocks in the affected file will show up as “missing” if the file system is checked with *fsck(C)*. Use *clri* only in emergencies and exercise extreme care.

Read and write permission is required on the specified *file-system* device. The inode becomes allocatable.

The primary purpose of this routine is to remove a file which for some reason appears in no directory. If you use *clri* to destroy an inode which does appear in a directory, track down the entry and remove it. Otherwise, when the inode is reallocated to some new file, the old entry will still point to that file. At that point removing the old entry will destroy the new file. The new entry will again point to an unallocated inode, so the whole cycle is likely to be repeated again and again.

See Also

fsck(C), *ncheck(C)*

Notes

If the file is open, *clri* is likely to be ineffective.



Name

`cmp` – Compares two files.

Syntax

```
cmp [-l] [-s] file1 file2
```

Description

Cmp compares two files and, if they are different, displays the byte and line numbers of the differences. If *file1* is `-`, the standard input is used.

The options are:

- `-l` Prints the byte number (decimal) and the differing bytes (octal) for each difference.
- `-s` Returns an exit code only, 0 for identical files, 1 for different files, and 2 for an inaccessible or missing file.

This command should be used to compare binary files; use *diff*(C) or *diff3*(C) to compare text files.

See Also

`comm`(C), `diff`(C), `diff3`(C)

Diagnostics

Exit code 0 is returned for identical files, 1 for different files, and 2 for an inaccessible or missing argument.



Name

`comm` – Selects or rejects lines common to two sorted files.

Syntax

`comm [ - [ 123 ] ] file1 file2`

Description

Comm reads *file1* and *file2*, which should be ordered in ASCII collating sequence (see *sort(C)*), and produces a three-column output: lines only in *file1*; lines only in *file2*; and lines in both files. The `- filename` means the standard input.

The **1**, **2**, or **3** flags suppress printing of the corresponding column. Thus *comm -12* prints only the lines common to the two files; *comm -23* prints only lines in the first file but not in the second; *comm -123* doesn't work and will print nothing.

See Also

`cmp(C)`, `diff(C)`, `sort(C)`, `uniq(C)`



Name

copy – Copies groups of files.

Syntax

copy [*option*] ... *source* ... *dest*

Description

The *copy* command copies the contents of directories to another directory. It is possible to copy whole file systems since directories are made when needed.

If files, directories, or special files do not exist at the destination, then they are created with the same modes and flags as the source. In addition, the super-user may set the user and group ID. The owner and mode are not changed if the destination file exists. Note that there may be more than one source directory. If so, the effect is the same as if the *copy* command had been issued for each source directory with the the same destination directory for each copy.

All of the options must be given as separate arguments and they may appear in any order even after the other arguments. The arguments are:

- a** Asks the user before attempting a copy. If the response does not begin with a “y”, then a copy is not done. This option also sets the **-ad** option.
- l** Uses links instead whenever they can be used. Otherwise a copy is done. Note that links are never done for special files or directories.
- n** Requires the destination file to be new. If not, then the *copy* command does not change the destination file. The **-n** flag is meaningless for directories. For special files an **-n** flag is assumed (i.e., the destination of a special file must not exist).
- o** If set then every file copied has its owner and group set to those of the source. If not set, then the file’s owner is the user who invoked the program.
- m** If set, then every file copied has its modification time and access time set to that of the source. If not set, then the modification time is set to the time of the copy.
- r** If set, then every directory is recursively examined as it is encountered. If not set then any directories that are found are ignored.
- ad** Asks the user whether an **-r** flag applies when a directory is discovered. If the answer does not begin with a “y”, then the directory is ignored.

- v** If the verbose option is set messages are printed that reveal what the program is doing.
- source** This may be a file, directory or special file. It must exist. If it is not a directory, then the results of the command are the same as for the *cp* command.
- dest** The destination must be either a file or directory that is different from the source.

If *source* and *destination* are anything but directories, then *copy* acts just like a *cp* command. If both are directories, then *copy* copies each file into the destination directory according to the flags that have been set.

See Also

cp(C), *umask*(C)

Notes

Special device files can be copied. When they are copied any data associated with the specified device is *not* copied.

The mode of the resulting files will be set using *umask* . The set-uid/gid fields are retained if the user was privileged. The sticky bit will be reset.

Name

cp – Copies files.

Syntax

cp *file1 file2*

cp *files directory*

Description

There are two ways to use the *cp* command. With the first way, *file1* is copied to *file2*. Under no circumstances can *file1* and *file2* be identical. With the second way, *directory* is the location of a directory into which one or more *files* are copied.

See Also

chmod(S), copy(C), cpio(C), ln(C), mv(C), rm(C)

Notes

Special device files can be copied. If the file is a named pipe, then the data in the pipe is copied to a regular file. Similarly, if the file is a device, then the file is read until the end-of-file is reached, and that data is copied to a regular file. It is illegal to copy a directory to a file.

Bugs

The *cp* command does not release file descriptors when an open attempt fails. This results in a different error message being produced after 20 failed opens.

The error message formats are not consistent.

Name

`cpio` – Copies file archives in and out.

Syntax

`cpio -o [ acBv ]`

`cpio -i [ Bcdmrtuv6 ] [ patterns ]`

`cpio -p [ adlmruv ] directory`

Description

Cpio-o (copy out) reads the standard input to obtain a list of pathnames and copies those files onto the standard output together with pathname and status information.

Cpio-i (copy in) extracts from the standard input (which is assumed to be the product of a previous *cpio -o*) the names of files selected by zero or more *patterns* given in the name-generating notation of *sh*(C). In *patterns* the special characters `?`, `*`, and `[...]` match the slash (`/`) character. The default for *patterns* is `*` (i.e., select all files).

Remember to escape special characters to prevent expansion by the shell.

Cpio-p (pass) copies out and in during a single operation. Destination pathnames are interpreted relative to the named *directory*.

The meanings of the available option flags are:

- a** Resets access times of input files after they have been copied.
- B** Inputs/outputs five blocks (1024 bytes each) per record (does not apply to the *pass* option; meaningful only with data directed to or from raw devices).
- d** Directories are created as needed.
- c** Writes header information in ASCII character form for portability.
- r** Interactively renames files. If the user types a null line, the file is skipped.
- t** Prints a table of contents of the input. No files are created.
- u** Copies unconditionally (normally an older file will not replace a newer file with the same name).

- v Verbose: causes a list of filenames to be printed. When used with the **t** option, the table of contents looks like the output of an *ls -l* command (see *ls(C)*).
- l Whenever possible, links files rather than copying them. Usable only with the **-p** option.
- m Retains previous file modification time. This option is ineffective on directories that are being copied.
- 6 Processes an old (UNIX System Sixth Edition format) file. Use only with the **-i** option.

Examples

The first example below copies the contents of a directory into an archive; the second duplicates a directory hierarchy:

```
ls | cpio -o >/dev/fd0  
  
cd olddir  
find . -print | cpio -pdl newdir
```

Or:

```
find . -print | cpio -oB >/dev/rfd0''
```

See Also

ar(CP), cpio(F), find(C)

Notes

Pathnames are restricted to 128 characters. If there are too many unique linked files, the program runs out of memory to keep track of them; thereafter, linking information is lost. Only the super-user can copy special files.

Name

`cron` - Executes commands at specified times.

Syntax

`/etc/cron`

Description

Cron executes commands at specified dates and times. Regularly scheduled commands can be specified according to instructions found in **crontab** files; users can submit their own **crontab** file via the *crontab* command. Commands which are to be executed only once may be submitted via the *at* command. Since *cron* never exits, it should only be executed once. This is best done by running *cron* from the initialization process through the `/etc/rc` file (see *init*(M)).

Cron only examines **crontab** files and *at* command files during process initialization and when a file changes. This examination reduces the overhead of checking for new or changed files at regularly scheduled intervals.

Files

<code>/usr/lib/cron</code>	Main <i>cron</i> directory
<code>/usr/lib/cron/log</code>	Accounting information
<code>/usr/spool/cron</code>	Spool area

See Also

`at`(C), `crontab`(C), `default`(M), `init`(M), `sh`(C)

Diagnostics

If you want to keep a log of all actions taken by *cron*, you must specify `CRONLOG=YES` in the `/etc/default/cron` file. If `CRONLOG=NO` (the default), no logging is done. Keeping the log is a user configurable option since *cron* usually creates huge log files. When `CRONLOB` is changed, *cron* will have to be stopped and restarted for this change to take effect.



Name

crontab – Manages user crontab files.

Syntax

```
crontab [file]  
crontab -r  
crontab -l
```

Description

Crontab copies the specified file, or standard input if no file is specified, into a directory that holds all users' **crontabs**. The **-r** option removes a user's **crontab** from the **crontab** directory. *Crontab* **-l** will list the **crontab** file for the invoking user.

A user is permitted to use *crontab* if his name appears in the `/usr/lib/cron/cron.allow` file. If that file does not exist, the `/usr/lib/cron/cron.deny` file is checked to determine if the user should be denied access to *crontab*. If neither file exists, only the super-user is allowed to submit a job. If either file is named **at.deny**, global usage is permitted. The allow/deny files consist of one user name per line.

A **crontab** file consists of lines of six fields each. The fields are separated by spaces or TABs. The first five are integer patterns that specify the following:

```
minute (0–59),  
hour (0–23),  
day of the month (1–31),  
month of the year (1–12),  
day of the week (0–6 with 0=Sunday).
```

Each of these patterns may be either an asterisk (meaning all legal values), or a list of elements separated by commas. An element is either a number, or two numbers separated by a minus sign (meaning an inclusive range). Note that the specification of days may be made by two fields (day of the month and day of the week). If both are specified as a list of elements, both are adhered to. For example, `0 0 1,15 * 1` would run a command on the first and fifteenth of each month, as well as on every Monday. To specify days by only one field, the other field should be set to `*` (for example, `0 0 * * 1` would run a command only on Mondays).

The sixth field of a line in a **crontab** file is a string that is executed by the shell at the specified times. A percent character in this field (unless escaped by `\`) is translated to a NEWLINE character. Only the first line (up to a `%` or end-of-line) of the command field is executed by the shell. The other lines are made available to the command as standard input.

The shell is invoked from your \$HOME directory with an `arg0` of `sh`. Users who desire to have their `.profile` executed must explicitly do so in the crontab file. *Cron* supplies a default environment for every shell, defining HOME, LOGNAME, SHELL(=/bin/sh), and PATH(=/bin:/usr/bin).

NOTE: Remember to redirect the standard output and standard error of the commands! If this is not done, any generated output or errors will be mailed to the user.

Files

/usr/lib/cron	Main <i>cron</i> directory
/usr/spool/cron/crontabs	Spool area
/usr/lib/cron/log	Accounting information
/usr/lib/cron/cron.allow	List of allowed users
/usr/lib/cron/cron.deny	List of denied users

See Also

`cron(C)`, `sh(C)`

Name

`csplit` – Splits files according to context.

Syntax

```
csplit [-s] [-k] [-f prefix] file arg1 [ . . . argn ]
```

Description

Csplit reads *file* and separates it into $n+1$ sections, defined by the arguments *arg1* . . . *argn*. By default the sections are placed in `xx00` . . . `xxn` (n may not be greater than 99). These sections get the following pieces of *file*:

- 00: From the start of *file* up to (but not including) the line referenced by *arg1*.
- 01: From the line referenced by *arg1* up to the line referenced by *arg2*.
- ⋮
- $n+1$: From the line referenced by *argn* to the end of *file*.

The options to *csplit* are:

- s** *Csplit* normally prints the character counts for each file created. If the **-s** option is present, *csplit* suppresses the printing of all character counts.
- k** *Csplit* normally removes created files if an error occurs. If the **-k** option is present, *csplit* leaves previously created files intact.
- f prefix** If the **-f** option is used, the created files are named *prefix00* . . . *prefixn*. The default is `xx00` . . . `xx n`.

The arguments (*arg1* . . . *argn*) to *csplit* can be a combination of the following:

- /rexp/** A file is to be created for the section from the current line up to (but not including) the line containing the regular expression *rexp*. The current line becomes the line containing *rexp*. This argument may be followed by an optional **+** or **-** some number of lines (e.g., `/Page/-5`).
- %rexp%** This argument is the same as **/rexp/**, except that no file is created for the section.
- lnno** A file is to be created from the current line up to (but not including) *lnno*. The current line becomes *lnno*.
- {num}** Repeat argument. This argument may follow any of the above arguments. If it follows a *rexp* type argument, that argument is applied *num* more times. If it follows *lnno*, the file will be split every *lnno* lines (*num* times) from that point.

Enclose all *regexp* type arguments that contain blanks or other characters meaningful to the shell in the appropriate quotation marks. Regular expressions may not contain embedded NEWLINES. *Csplit* does not affect the original file; it is the users responsibility to remove it.

Examples

```
csplit -f cobol file '/procedure division/' /par5./ /par16./
```

This example creates four files, **cobol00** . . . **cobol03**. After editing the “split” files, they can be recombined as follows:

```
cat cobol0[0-3] > file
```

Note that this example overwrites the original file.

```
csplit -k file 100 {99}
```

This example would split the file at every 100 lines, up to 10,000 lines. The **-k** option causes the created files to be retained if there are less than 10,000 lines; however, an error message would still be printed.

```
csplit -k prog.c '%main(%' '/' +1' {20}
```

Assuming that **prog.c** follows the normal C coding convention of ending routines with a **}** at the beginning of the line, this example will create a file containing each separate C routine (up to 21) in **prog.c**.

See Also

ed(C), regex(S), sh(C)

Diagnostics

Self-explanatory except for:

arg – out of range

which means that the given argument did not reference a line between the current position and the end of the file.

Name

cu – Calls another XENIX system.

Syntax

cu [*-s speed*] [*-a acu*] [*-l line*] [*-h*] [*-o|-e*] *telno*

cu [*-s speed*] *-l line* [*-h*] [*-o|-e*] *dir*

Description

Cu “calls up” another XENIX system through a modem or a direct serial connection. It also controls the transmission and reception of data and programs during the call. *Cu* looks at each line in the */usr/lib/uucp/L-devices* file until it finds a line that matches the options given in the command line. If it finds an appropriate line, it will attempt to make a connection. If it cannot find the proper line, *cu* quits.

The options are:

-speed

Specifies the transmission speed. 1200 baud is the default value. Other speeds available are 110, 150, 300, 1200, 2400, 4800 and 9600 baud. Directly connected lines may be set to other speeds. Most modems are restricted to 300 and 1200 baud.

-acu

Specifies the device name of the ACU (automatic calling unit) device. When used with the *-l* option, overrides the search for the first available ACU with the right speed.

-line

Specifies the device name of the communications line. When used with the *-a* option, overrides the search for the first available ACU with the right speed.

-h

Emulates local echo. This feature supports calls to systems that expect half-duplex mode terminals.

-e

Specifies that even-parity data is to be generated for data sent to the remote system.

-o

Specifies that odd-parity data is to be generated for data sent to the remote system.

Telno is the telephone number of the remote system. A comma indicates a delay at appropriate places, for example, to wait for a secondary dial tone when calling from an internal phone system. For directly connected lines, the string “dir” is used instead of *telno*. Direct lines require a line to be specified, but no ACU. See the Examples later in this section for sample command lines.

After making the connection, *cu* runs as two processes: *transmit* and *receive*. The *transmit* process reads data from the standard input and, except for lines beginning with a tilde (~), passes it to the remote system. The *receive* process accepts data from the remote system and, except for lines beginning with a tilde, passes it to the standard output. Normally, an automatic XON/XOFF (DC3/DC1) protocol controls input from the remote system so the buffer is not overrun. Lines beginning with a tilde have special meanings.

The *transmit* process interprets lines beginning with a tilde as follows:

- ~. Terminates the conversation.
- ~! Escapes to an interactive shell on the local system.
- ~!cmd... Runs *cmd* on the local system (via *sh -c*).
- ~\$cmd... Runs *cmd* locally and sends its output to the remote system.
- ~%take remote [local]
Copies file *remote* (on the remote system) to file *local* on the local system. If *local* is omitted, the *remote* filename is used in both places. Use of this line requires the existence of *echo*(C) and *cat*(C) on the remote system. If tabs are to be copied without expansion, *stty tabs* mode should be set on the remote system.
- ~%put local [remote]
Copies file *local* (on the local system) to file *remote* on the remote system. If *local* is omitted, the *remote* filename is used in both places. Use of this line requires the existence of *stty*(C) and *cat*(C) on the remote system. It also requires that the current ERASE and KILL characters on the remote system be identical to the current ones on the local system. Backslashes are inserted at appropriate places.
- ~... Sends the line ~... to the remote system.
- ~%nostop
Turns off the XON/XOFF input control protocol for the remainder of the session. This is useful if the remote system is one which does not respond properly to the XON/XOFF characters.

The *receive* process normally copies data from the remote system to its standard output. A line from the remote system that begins with `~>` diverts the output to a file. Data is appended to a file if `~>>` is used. The diversion is terminated by a trailing `~>`. The complete sequence is:

```
~>[>]: file
      zero or more lines to be written to file
~>
```

Examples

A sample command line might be:

```
cu -s2400 -l/dev/cua0 4479801
```

Where “-s2400” indicates a line speed of 2400 baud, and “-l/dev/cua0” is the device name of the communications line. The system looks in the file **L-devices** for a line with these characteristics. “4479801” is the phone number of the remote system. For a directly connected line:

```
cu -l/dev/tty01 dir
```

“Dir” indicates a direct line connection.

To dial out of an internal phone system, such as a computerized branch exchange (CBX):

```
cu -s2400 -l/dev/cua0 9,4479801
```

If your system does not automatically wait for an outgoing dial tone, use several commas for an extended delay:

```
cu -s2400 -l/dev/cua0 9,,,4479801
```

Files

/usr/lib/uucp/L-devices	Device information
/usr/spool/uucp/LCK..(tty-device)	Lockout mechanism
/usr/lib/uucp/dial	Dialer program
/dev/null	

See Also

cat(C), echo(C), stty(C), -ty(M)

Diagnostics

Exit code is zero for normal exit, nonzero (various values) otherwise.

Notes

There is an artificial slowing of transmission by *cu* during the *~%put* operation so that loss of data is unlikely.

ASCII files only can be transferred using *~%take* or *~%put*; binary files cannot be transferred.

The modem line or direct serial line on the local system used to make the call must have logins disabled. See *disable(C)*. The modem line or direct serial line on the remote system used to receive the call must have logins enabled. See *enable(C)*.

Name

`date` – Prints and sets the date.

Syntax

`date [ mmddhhmm[yy] ] [ +format ]`

Description

If no argument is given, or if the argument begins with `+`, the current date and time are printed. Otherwise, the current date is set. The first *mm* is the month number; *dd* is the day number in the month; *hh* is the hour number (24-hour system); the second *mm* is the minute number; *yy* is the last 2 digits of the year number and is optional. For example:

```
date 10080045
```

sets the date to Oct 8, 12:45 AM. The current year is the default if no year is mentioned. The system operates in GMT. *Date* takes care of the conversion to and from local standard and daylight time.

If the argument begins with `-`, the output of *date* is under the control of the user. The format for the output is similar to that of the first argument to *printf*(S). All output fields are of fixed size (zero padded if necessary). Each field descriptor is preceded by a percent sign (`%`) and will be replaced in the output by its corresponding value. A single percent sign is encoded by doubling the percent sign, i.e., by specifying `“%%”`. All other characters are copied to the output without change. The string is always terminated with a NEWLINE character.

Field Descriptors:

- n** Inserts a NEWLINE character
- t** Inserts a TAB character
- m** Month of year – 01 to 12
- d** Day of month – 01 to 31
- y** Last 2 digits of year – 00 to 99
- D** Date as *mm/dd/yy*
- H** Hour – 00 to 23

- M Minute – 00 to 59
- S Second – 00 to 59
- T Time as *HH:MM:SS*
- j Julian date – 001 to 366
- w Day of the week – Sunday = 0
- a Abbreviated weekday – Sun to Sat
- h Abbreviated month – Jan to Dec
- r Time in AM/PM notation

Example

The line:

```
date '+DATE: %m/%d/%y%nTIME: %H:%M:%S'
```

generates as output:

```
DATE: 08/01/76
TIME: 14:45:05
```

Diagnostics

- no permission* You aren't the super-user and you are trying to change the date.
- bad conversion* The date set is syntactically incorrect.
- bad format character* The field descriptor is not recognizable.

Name

dc – Invokes an arbitrary precision calculator.

Syntax

dc [*file*]

Description

Dc is an arbitrary precision arithmetic package. Ordinarily it operates on decimal integers, but you may specify an input base, output base, and a number of fractional digits to be maintained. The overall structure of *dc* is a stacking (reverse Polish) calculator. If an argument is given, input is taken from that file until its end, then from the standard input. The following constructions are recognized:

number

The value of the number is pushed on the stack. A number is an unbroken string of the digits 0–9. It may be preceded by an underscore (`_`) to input a negative number. Numbers may contain decimal points.

`+ - / * % ^`

The top two values on the stack are added (+), subtracted (–), multiplied (*), divided (/), remaindered (%), or exponentiated (^). The two entries are popped off the stack; the result is pushed on the stack in their place. Any fractional part of an exponent is ignored.

sx The top of the stack is popped and stored into a register named *x*, where *x* may be any character. If the *s* is capitalized, *x* is treated as a stack and the value is pushed on it.

lx The value in register *x* is pushed on the stack. The *x* register is not altered. All registers start with zero value. If the *l* is capitalized, register *x* is treated as a stack and its top value is popped onto the main stack.

d The top value on the stack is duplicated.

p The top value on the stack is printed. The top value remains unchanged. **P** interprets the top of the stack as an ASCII string, removes it, and prints it.

f All values on the stack are printed.

q Exits the program. If executing a string, the recursion level is popped by two. If **q** is capitalized, the top value on the stack is popped and the string execution level is popped by that value.

- x** Treats the top element of the stack as a character string and executes it as a string of *dc* commands.
- X** Replaces the number on the top of the stack with its scale factor.
- [...]** Puts the bracketed ASCII string onto the top of the stack.
- <x >x =x**
The top two elements of the stack are popped and compared. Register *x* is evaluated if they obey the stated relation.
- v** Replaces the top element on the stack by its square root. Any existing fractional part of the argument is taken into account, but otherwise the scale factor is ignored.
- !** Interprets the rest of the line as a XENIX command.
- c** All values on the stack are popped.
- i** The top value on the stack is popped and used as the number radix for further input.
- I** Pushes the input base on the top of the stack.
- o** The top value on the stack is popped and used as the number radix for further output.
- O** Pushes the output base on the top of the stack.
- k** The top of the stack is popped, and that value is used as a nonnegative scale factor; the appropriate number of places are printed on output, and maintained during multiplication, division, and exponentiation. The interaction of scale factor, input base, and output base will be reasonable if all are changed together.
- z** The stack level is pushed onto the stack.
- Z** Replaces the number on the top of the stack with its length.
- ?** A line of input is taken from the input source (usually the terminal) and executed.
- ;;** Used by *bc* for array operations.

Example

This example prints the first ten values of $n!$:


```
[la1+dsa*pla10>y]sy
0sa1
lyx
```

See Also

bc(C)

Diagnostics

<i>x is unimplemented</i>	The octal number <i>x</i> corresponds to a character that is not implemented as a command
<i>stack empty</i>	Not enough elements on the stack to do what was asked
<i>Out of space</i>	The free list is exhausted (too many digits)
<i>Out of headers</i>	Too many numbers being kept around
<i>Out of pushdown</i>	Too many items on the stack
<i>Nesting Depth</i>	Too many levels of nested execution

Notes

Bc is a preprocessor for *dc*, providing infix notation and a C-like syntax which implements functions and reasonable control structures for programs. For interactive use, *bc* is preferred to *dc*.



Name

dd – Converts and copies a file.

Syntax

dd [*option=value*] ...

Description

Dd copies the specified input file to the specified output with possible conversions. The standard input and output are used by default. The input and output block size may be specified to take advantage of raw physical I/O. A record is equivalent to whatever the current block size is (as specified or the default.)

<i>Option</i>	<i>Value</i>
if=file	Input filename; standard input is default
of=file	Output filename; standard output is default
ibs=n	Input block size <i>n</i> bytes (default 1024)
obs=n	Output block size (default 1024)
bs=n	Sets both input and output block size, superseding <i>ibs</i> and <i>obs</i> ; also, if no conversion is specified, it is particularly efficient since no in-core copy needs to be done
cbs=n	Conversion buffer size
skip=n	Skips <i>n</i> input records before starting copy
seek=n	Seeks <i>n</i> records from beginning of output file before copying
count=a	Copies only <i>n</i> input records
conv=ascii	Converts EBCDIC to ASCII
conv=ebcdic	Converts ASCII to EBCDIC
conv=ibm	Slightly different map of ASCII to EBCDIC
conv=lcase	Maps alphabetics to lowercase
conv=ucase	Maps alphabetics to uppercase

conv=swab Swaps every pair of bytes

conv=noerror
Does not stop processing on an error

Option *Value*

conv=sync Pads every input record to *ibs*

conv="... , ..."
Several comma-separated conversions

Where sizes are specified, a number of bytes is expected. A number may end with **k**, **b**, or **w** to specify multiplication by 1024, 512, or 2 respectively; a pair of numbers may be separated by **x** to indicate a product.

Cbs is used only if *ascii* or *ebcdic* conversion is specified. In the former case *cbs* characters are placed into the conversion buffer, converted to ASCII, and trailing blanks trimmed and newline added before sending the line to the output. In the latter case ASCII characters are read into the conversion buffer, converted to EBCDIC, and blanks added to make up an output record of size *cbs*.

After completion, *dd* reports the number of whole and partial input and output blocks.

Examples

This command reads an EBCDIC tape, blocked ten 80-byte EBCDIC card images per record, into the ASCII file, **outfile**:

```
dd if=/dev/rmt0 of=outfile ibs=800 cbs=80 conv=ascii,lcase
```

Note the use of raw magtape. *Dd* is especially suited to I/O on raw physical devices because it allows reading and writing in arbitrary record sizes.

See Also

copy(C), cp(C), tar(C)

Diagnostics

f+p records in(out) Numbers of full and partial records read(written)

Notes

The ASCII/EBCDIC conversion tables are taken from the 256-character standard in the CACM Nov, 1968. The *ibm* conversion corresponds better to certain IBM print train conventions. There is no universal solution.

NEWLINES are inserted only on conversion to ASCII; padding is done only on conversion to EBCDIC.

